Lab 8: Vision-Based Object Detection for Robotic Manipulation

EE 471: Vision-Based Robotic Manipulation Laboratory

December 3, 2024

Fall 2024

Group Members: Ashik Islam,
Dmitri Dobrynin

1. Introduction

In this lab, we aimed to create a complete vision-guided robotic manipulation system by developing a robust pick-and-place application where the robot arm identifies colored spherical objects with vision from the Intel Realsense (D435) camera and sorts them into designated locations by their colors. This required us to develop a new algorithm to detect the objects in real time and determine their pose in the robot frame. Additionally, we combined and utilized many of the algorithms we had developed in previous labs.

The application we implemented worked as follows: the Realsense camera was fixed to provide a clear view of the robot's base plate. Multiple colored balls were placed on the base plate (which is defined as the workspace) alongside four bins next to the plate, each designated for one color. The colors were red, orange, yellow, and blue. The robot detected and identified the balls and their colors and sorted them into their corresponding bins by grasping and placing them appropriately. It continued sorting until no more balls remained on the base plate while ignoring any balls off the plate, even if they were within the camera's view or the robot's reach. The base plate was continuously monitored so that if additional balls were placed, the robot resumed sorting. The lighting conditions remained relatively consistent throughout the process.

2. Development and Implementation

From previous labs, we already have implementations for successfully sending the end-effector to a given reachable pose in the base frame. We also have implementations for converting the position in the image frame to the camera frame to the robot base frame. We want to command the end-effector to grab the spherical objects, which are soft balls with a 15mm radius; The balls can be red, orange, yellow, or blue. To find the pose of the balls in the robot base frame, we need to detect them, their colors, and their position in the image frame, then convert them positively to the robot base frame.

2.1. Vision-Based Object Detection

To detect colored balls, a method called detect_colored_spheres() was developed. This method takes an image path as a string and returns a Python list of tuples that hold color, centroid, and radius information of each sphere in the image. This method imports or reads an image saved to the image path passed in and converts the image to HSV color space. The method adds a gaussian blur to filter noise and has color thresholds to create masks of each color for image segmentation. The result is a combined mask that only has the detected spheres present in the image.

The next step is to perform image enhancement. The contrast in the image is increased to make detection of the contours easier. The image is converted to grayscale and another Gaussian blur filter is applied to smooth out the image. Image segmentation is done with Otsu thresholding and is followed by detecting the contours. Then for each contour, we find the area and perimeter of each contour. From this information, the circularity is calculated, which is used with the area to

determine if the detected contour is a sphere. If the contour meets the conditions, the centroid and radius of the sphere is calculated. Now we have the position of the balls in image frame. The contour of the circles and their centroids are overlaid over the image on the laptop screen to visually confirm the ball detections. The HSV value of the centroid pixel of each circle is tested based on the color thresholds defined above to determine the sphere color. The name of the color is added to the original image. Each sphere's centroid, radius and color are recorded in a list. One of the challenges we encountered with vision-based object detection was how to handle the drastic variations in light around the lab station. Camera position and orientation also affected the ability of the method to detect the colored spheres. The method developed was fairly robust but could not handle completely different lighting conditions. The performance was best in the evening, as the images used to tune the color thresholds were taken during that time. A significant amount of time was spent on tuning the color thresholds, and we had to take many steps to enhance the image to improve performance. However, we were able to achieve object detection for spheres that were directly side-by-side, even if their sides were touching.

2.2. Object Localization

For object localization, we chose to use the perspective-n-point (PnP) algorithm, which we modified from the get_tag_pose() method for AprilTags to work with spheres in the get_sphere_pose() method. The get_sphere_pose() method takes in the centroid coordinates and radius of the sphere in the image frame, the camera intrinsics, and the physical radius of the ball of 15 mm. The method returns the translation vector and rotation matrix required to translate the spheres from the image to the camera frame. The method first defines 4 3D model points on the spheres using the physical radius size. Using the centroid and radius in the image frame, 4 corresponding image points on the spheres are created. The order of the points in the image frame and 3D model must match. The camera matrix is then constructed from the camera intrinsics. With this, we have enough information to solve the PnP, obtaining the rotation matrix and translation vector. We chose not to use geometric analysis of the spheres to do corrections for the PnP algorithm. While the geometric correction would have been the best solution, we opted to make manual adjustments to the localization of the spheres through tuning and experimentation. One challenge we encountered with the localization was the positioning of the camera. We found that it was best to position the camera directly opposite the robot arm close to the base frame, at a height where the camera could see the entire workspace from above (see Figure 2). This resulted in the best localization performance and required trying various setups to settle on a good camera position.

2.3. Vision-Guided Robotic Sorting System Implementation

In this section, we will describe the system architecture for the pick-and-place sorting system. Before running the pick-and-place sorting script, we must first run the camera calibration script developed in lab 6 under lab6_2.py, which gives us the transformation matrix used to convert from the camera to the robot base frame. To obtain this transformation, we do point registration

using the Kabsch algorithm. In the main script, the transformation matrix is loaded, and various constants and variables are defined. These include the physical radius size of the spheres (15 mm), the home position and bin locations in the task space, two separate trajectory times of 1 and 2 seconds, and the number of intermediate waypoints for a trajectory (998). The robot is then commanded to move to the home position. For this motion, no trajectory generation is applied, we simply convert the home position to a joint configuration using inverse kinematics and write the robot's joints. Once the robot is in the home position, it scans the workspace and captures image frames that will be given to the detect_colored_spheres() method to search for spheres. The first frame is poor in quality when the camera starts up, so it is always ignored. For each sphere that is detected, we call the get_sphere_pose() method to get the pose in the camera frame, which is transformed to the robot frame using the transformation matrix found previously. With all the spheres detected, we mask the workspace to only pick-up balls located on the base plate. Knowing the physical dimensions of the workspace, we filter out any spheres detected outside of the workspace so that the robot does not try to sort them. The physical dimensions were taken from a previous lab and were chosen as 15 mm to 250 mm in the x-direction and -125 mm to 125 mm in the y-direction.

Now we are left with a list of detected spheres. If it is empty, the robot does nothing and continues collecting image frames, repeating the steps above and waiting in the home position until a sphere is detected. If a sphere is detected, it will begin sorting. The sorting was done based on how the balls were ordered in the detected spheres list. This means we did not prioritize certain colors or define an order for sorting. For the first detected sphere in the list, the color is saved and the pose in the robot frame is obtained in the same way as it was for the workspace masking. Next, two poses are defined: the pose of the end-effector to grab the sphere and the pose of the end-effector above the sphere. For these poses, the z-values were hardcoded, as we know the spheres are always on a flat surface, and the orientation angle of the pitch was -90 degrees for both poses. We also made a slight correction to the x-position to account for localization errors. Using if statements, further correction is done based on the position in the workspace. Near the edges, the localization was less accurate, so these if statements correct that discrepancy. We also define a pose for the robot arm to move to in the center of the workspace. Based on the color of the detected sphere, we save the pose for the bin the ball should be placed in. We now have everything we need to begin picking up and sorting the spheres.

For the motion of the robot arm, we chose to do trajectory planning by generating quintic polynomial trajectories. Using quintic polynomial trajectories, we were able to get linear motion between poses and we had control over the initial and final velocity and acceleration, setting both to at the start and end of a trajectory. We created a method called move_gripper_to() which took a set of poses to generate trajectories between, the trajectory time, and the number of intermediate points. This method was called every time we want the robot to move to a new position. The robot would first move to the center from the home position. Next, it moves above the sphere it is trying to sort and opens the gripper. Next, it moves down to grab the sphere,

closes the gripper and returns to the position above where the ball was located. It returns to the center position, moves above the target bin and releases the ball. Finally, it closes the gripper again, moves back to the center, and returns to the home position. The center position was added to prevent the robot from moving too quickly between poses. If the robot generated a trajectory near the base or a ball was positioned far away from the bin, the joints would move very rapidly and we risked damaging the robot, so the center position was our solution to increase the safety for the arm. Additionally, if a sphere was positioned in way that the arm could not reach the desired end-effector pose to grab it, the script throws an exception, and the robot does not proceed further. This is another safety precaution to avoid damage to the robot. Since the robot returns to the home position and continues to scan the workspace, if it fails to pick up one of the balls, it will detect it again and can attempt to sort it again. On the following page, Figure 1 shows the flowchart of the system.

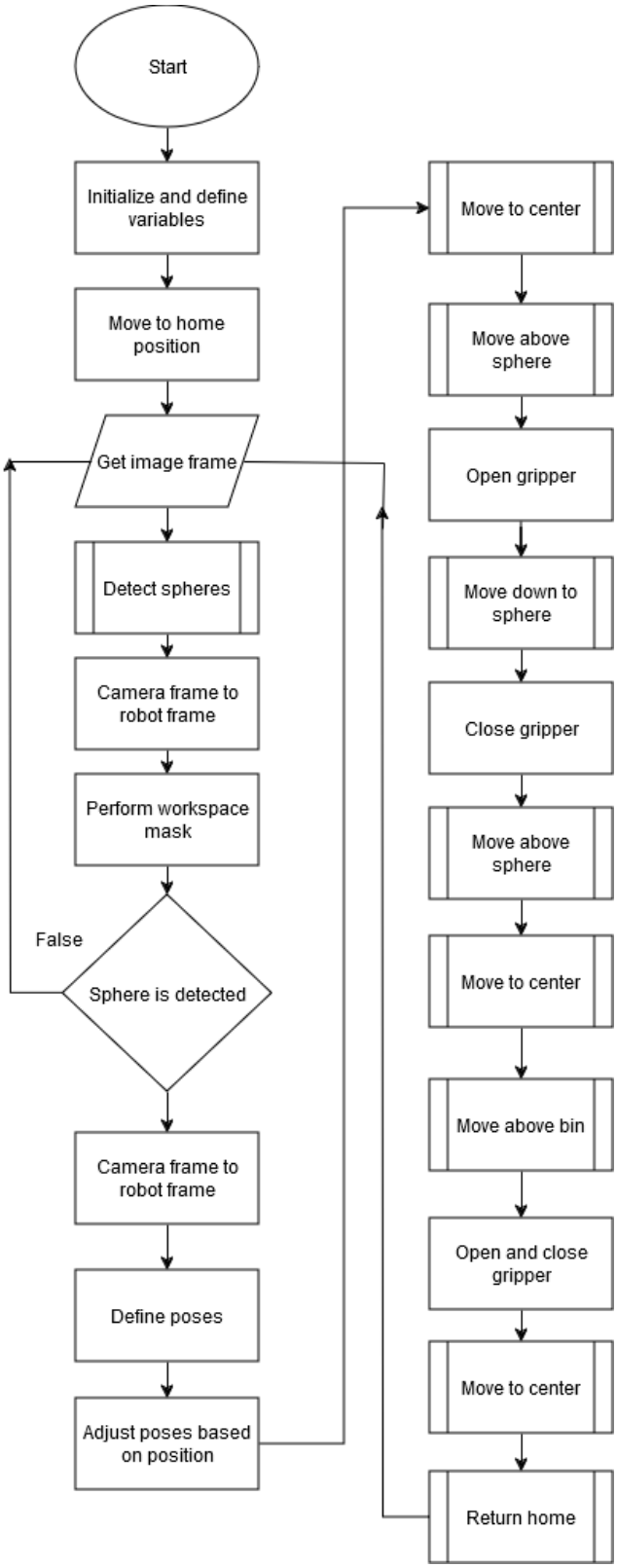

Figure 1: Flowchart for pick-and-place system

3. Results

3.1. Visual Object Detections

The function of our detect_colored_spheres() is illustrated with an example below. First, the Realsense camera is set up with the robot's baseplate in view of the camera as shown in Figure 2. Next, camera-robot calibration is done.

Figure 2: Image of the workspace and setup

The calibration process is given a set of 12 points in the robot base frame; it finds the corresponding points in camera frames and using the 3D point registration via the Kabsch algorithm, it determines the transformation matrix between the camera frame and robot base frame. Figure 3 shows the successful transformation of camera frame points to robot base frame points for this configuration. Figure 4 shows the computed transformation matrix and the Root Mean Square Error (RSME); lower RSME values indicate a better fit between the transformed point and their target locations. Our RSME value is only 1.42 mm, which is an acceptable value as it is a very insignificant error in our application. In other words, our camera-to-robot frame conversion is very accurate.

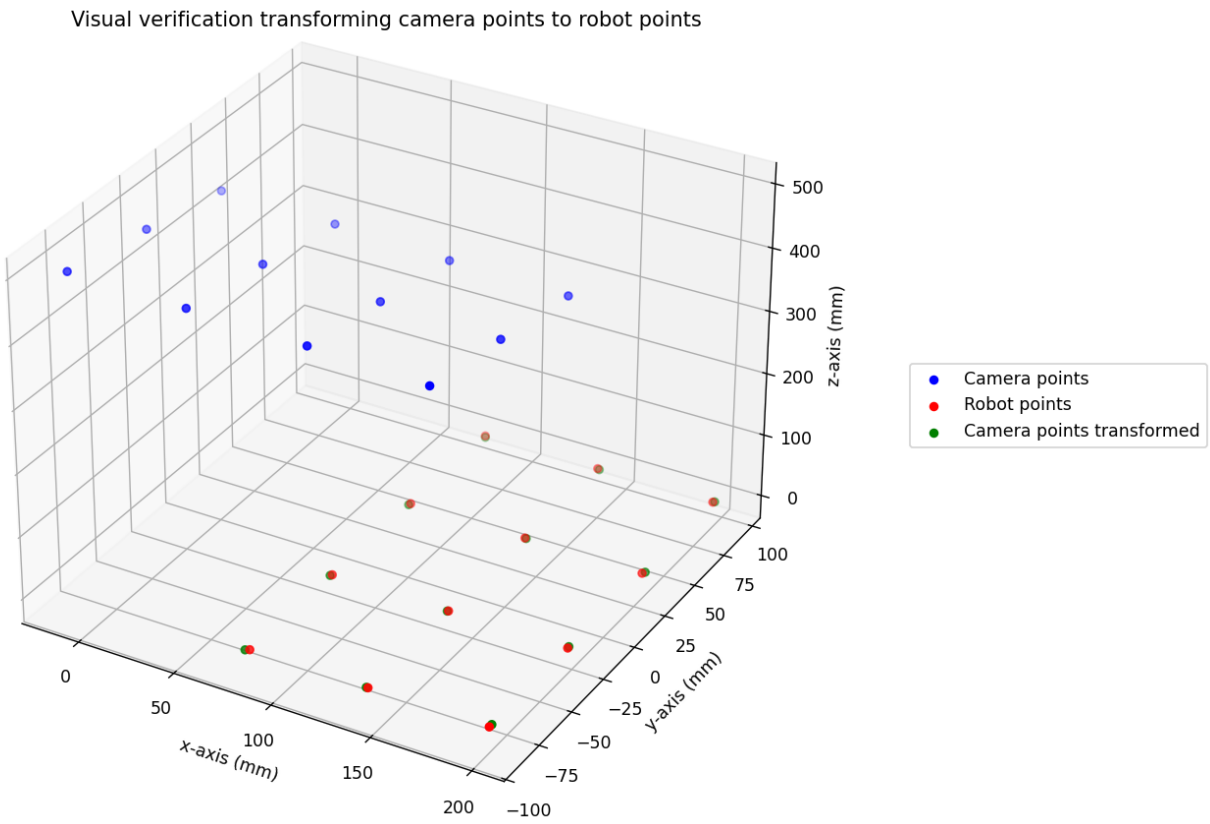

Figure 3: Camera calibration transformation results

```
Transformation matrix from camera to robot, T_cam_robot:
 [[-3.12407803e-02  8.84225133e-01 -4.66014943e-01  3.81322107e+02]
 [ 9.96380774e-01 -9.32504367e-03 -8.44890307e-02 -3.37392730e+01]
 [-7.90529341e-02 -4.66967833e-01 -8.80733601e-01  3.97895646e+02]
 [ 0.00000000e+00  0.00000000e+00  0.00000000e+00  1.00000000e+00]]
RMSE(mm): 1.4227049068413915
```

Figure 4: Camera calibration transformation matrix and error (RMSE)

Now that our camera-robot calibration is done, a camera image such as Figure 5 (without the annotations) is passed to the detect_colored_spheres(), which detects all the spheres in the image along with their colors, centroids(cx, cy), and radii in the image frame. The detections are overlaid on the image as shown in Figure 5 for visual verification. For this specific image, the values computed by the method and the actual values are tabulated Table 1. This method produced the exact values correctly with no error. The actual values were measured using an image inspection tool. We can now conclude that our implementation of detect_colored_spheres() accurately detects the colored spheres and their properties.

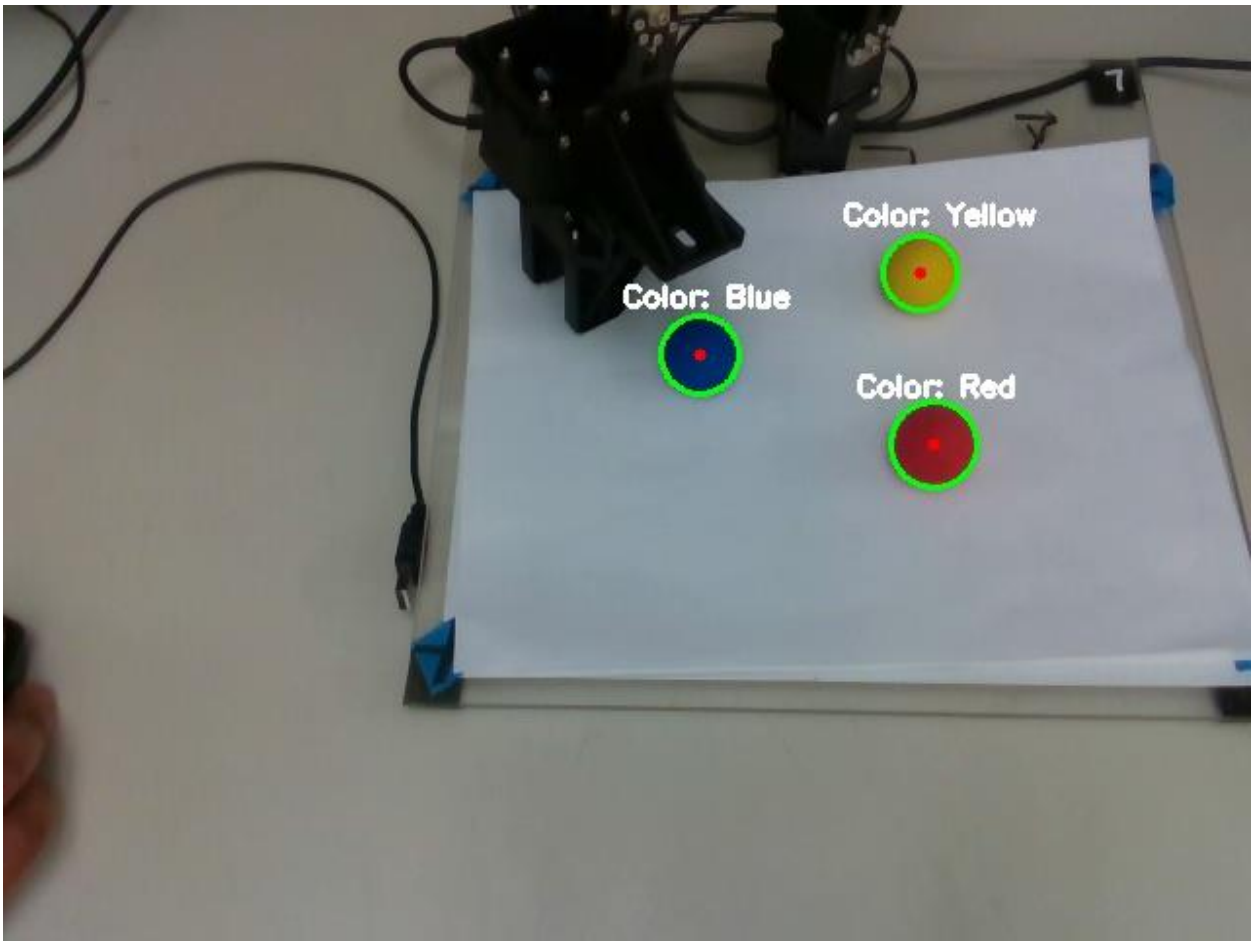

Figure 5: Processed image with contours, centroids, and colors added to image to show detections

Table 1: Sphere detections for the image in Figure 5.

Computed			Actual		
Color	Centroids (cx, cy) [px]	Radius [px]	Color	Centroids (cx, cy) [px]	Radius [px]
Red	(477, 225)	22	Red	(477, 225)	22
Blue	(357, 179)	20	Blue	(357, 179)	20
Yellow	(470, 137)	19	Yellow	(470, 137)	19

3.2. Object Localization

Now that we have accurate sphere centroids and their radii in the image frame, we use the PnP algorithm described in Section 2.2 to convert the 2D image coordinates into 3D robot base frame coordinates. Table 2 lists the 3D robot base frame coordinates computed by our implementation of the PnP algorithm from the frame in Figure 6 and compares them to the measured values. The measured values were measured using a ruler.

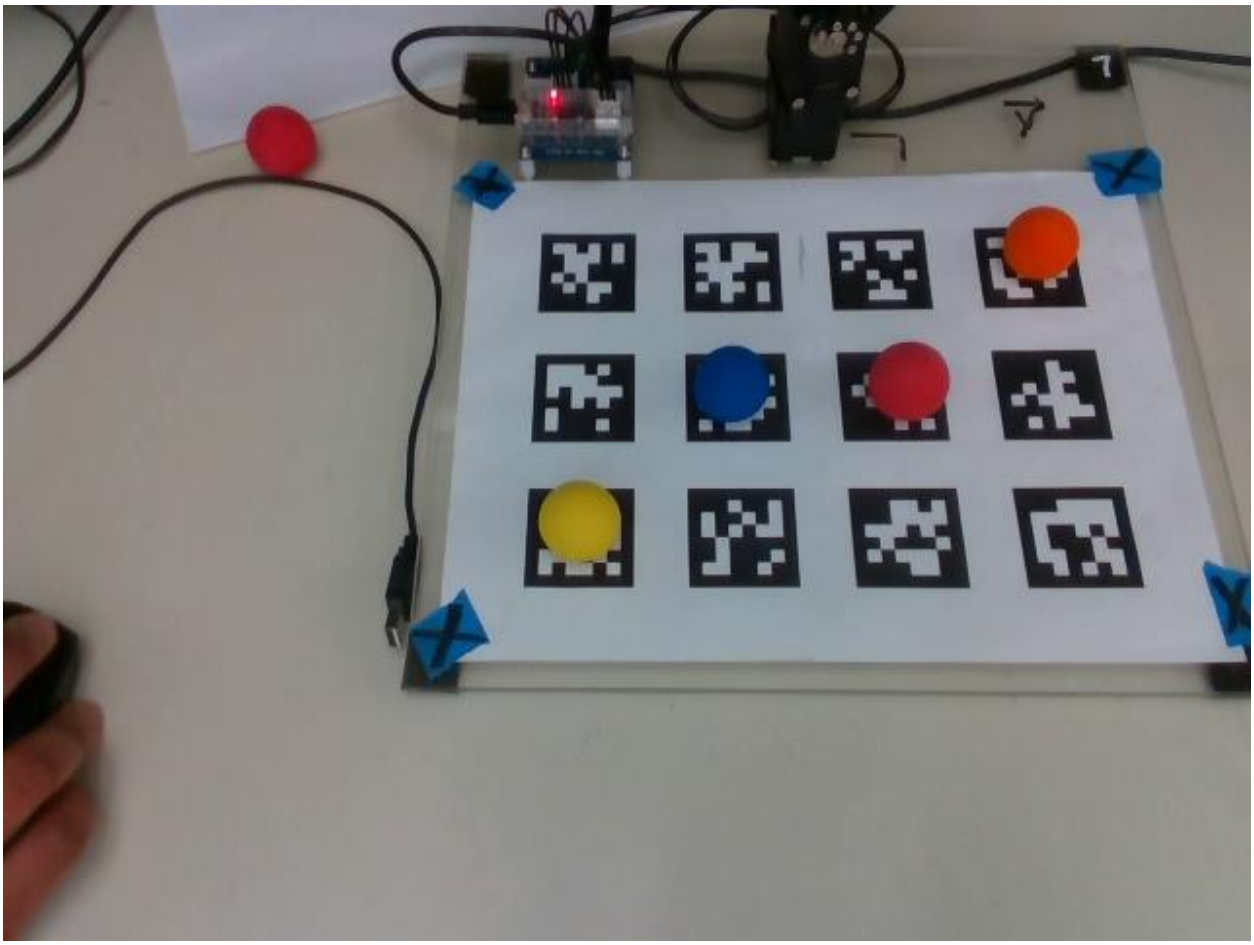

Figure 6: Image of the setup for measurement collection

Table 2: Computed vs. measured positions of the centroid of a ball (in the robot frame)

Computed position x, y, z [mm]	Measured position x, y, z [mm]	Absolute error x, y, z [mm]
63.6, 93.2, 5.5	79, 93, 15	15.4, 0.2, 9.5
131.9, -31.9, 9.6	139, -27, 15	7.1, 4.9, 5.4
140.1, 32.8, 23.1	139, 33, 15	1.1, 0.2, 8.1
190.1, -87.5, 9.3	199, -87, 15	8.9, 0.5, 5.7

There is a noticeable error present in 3D robot base frame coordinates. This error could have been lowered using geometric analysis mentioned in section 2.2. As stated earlier, we opted to make manual adjustments to the localization. Through iterative testing, we found that the adjustments described in Figure 7 can be made to the sphere centroids coordinates in robot base frame for a sufficiently accurate pose estimation for the end-effector to reliably grasp the spheres.

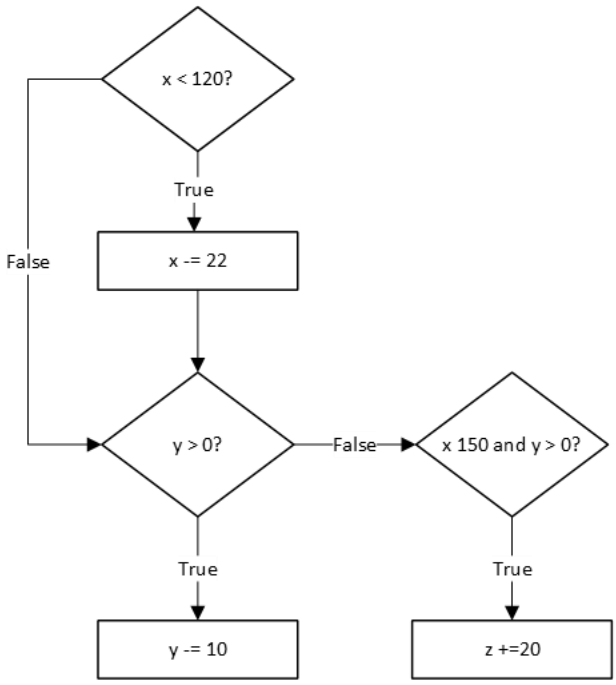

Figure 7: Image of the setup for measurement collection

4. Discussion

We successfully developed a complete vision-guided robotic sorting system that successfully detects, identifies and manipulates colored spherical objects to sort them into designated locations based on their color. We employed a few image processing techniques to a live camera feed to enhance sphere detection through contour analysis to acquire the sphere's color, centroids and radius in the image frame. Using this information and a camera-to-robot base frame transformation matrix obtained via camera-robot calibration, the pose of the spheres in the robot base frame was estimated. We observed an error in the pose estimation which was manually adjusted based on the x and y coordinates of the pose. Trajectory planning technique was used to command the robot to grasp the spherical object because of its accurate positioning.

During the implementation of trajectory planning, we ran into an issue of not giving the robot enough time to execute a trajectory; we quickly solved this issue by calculating the time between each intermediate trajectory. Since there were trajectories of varying Euclidean distance, we chose to use a short trajectory time (1 second) and a long trajectory time (2 seconds) for consistent and smooth motion throughout the trajectories. Additionally, we commanded the robot to go to 117 mm above the center of the base plate so ensure that the base joint never rotates rapidly when the end-effect needs to go from the orange bin to the home position.

The largest challenge we encountered was ensuring that the pose estimation was accurate for the spherical objects everywhere in the workspace. We realized a quick and simple way to ensure that the pose estimation was accurate enough for the purpose of this application was to adjust the pose based on its x and y coordinates. In the future, a geometric analysis of the sphere center correction can be done for more accurate pose estimation.

Another challenge we encountered was ignoring the spherical objects outside of the workspace. We found an efficient and elegant way to successfully ignore any spherical object outside of the workspace by simply checking to see the calculated robot frame sphere pose; if the pose is outside of the defined workspace, we simply ignore that sphere and move onto the next detected sphere.

5. Conclusion

This final lab was a culmination of this course. We were able to combine parts of almost every lab to create a system that successfully picks up and sorts the colored spheres. We utilized inverse and forward kinematics, trajectory generation, computer vision and image processing techniques, point registration with the Kabsch algorithm, and the PnP algorithm for pose estimation. This lab tied together many concepts by putting them all into action to create a finished product. This final project gives us a project to add to our portfolios and prepares us for work in a robotics engineering role.